UPANZI
DIGITAL PUBLIC INFRASTRUCTURE
NETWORK

Pepper Robot Tour for Digital Experience Center

D3.1 System Architecture

Due date: **28/02/2026**
Submission Date: **28/02/2026**
Revision Date: **15/08/2026**

Start date of project: **01/07/2026**

Duration: **12 months**

Responsible Person: **Yohannes Haile**

Revision: **1.9**

Project funded by Upanzi Network Dissemination Level		
PU	Public	**PU**
PP	Restricted to other programme participants	
RE	Restricted to a group specified by the consortium	
CO	Confidential, only for members of the consortium	

Executive Summary

This deliverable specifies the DEC system architecture in detail: the component subsystems, the modules (i.e., ROS2 nodes) comprising each subsystem, and the information exchanged between subsystems and modules. This includes a specification of the data that are input to each module, the data that are output from each module, and the data that are used to control the operation of the module, including the manner in which this data is made available or accessed: through ROS2 topics, services, actions, or through configuration and data files.

The system implements the Pepper Robot Tour at the Digital Experience Center (DEC) of the Upanzi Network at Carnegie Mellon University Africa. It runs on **ROS2 Humble** and follows a modular, component-based software engineering approach using the publish-subscribe model and the ROS2 Managed Node (Lifecycle) pattern for graceful start-up and shutdown.

The architecture identifies two primary operational subsystems. The **Robot Sensing subsystem** (WP4) encompasses perception and attention modules: YOLO-based person detection, SixDrepNet-based face and mutual-gaze detection, MiVOLO-based age/gender estimation, VAD-aware speech recognition with an action-server interface, sound-source localization, and a saliency-driven overt attention controller. The **Robot Behaviors subsystem** (WP5) encompasses the action, coordination, and navigation modules: the BehaviorTree.CPP v4 Behavior Controller that orchestrates the full tour, a RAG-based Conversation Manager, a gesture execution module, an animated background behavior module, a text-to-speech module, and a navigation stack comprising a mapping/localization layer (RTAB-Map, SLAM Toolbox, or FAST-LIO lidar-inertial odometry over a Unitree L2 + RealSense sensor rig) and a Nav2 layer for path planning, obstacle avoidance, and a collision-monitor safety layer.

All inter-node communication is defined using custom `dec_interfaces` action, message, and service types together with standard ROS2 message types. Robot actuator commands use `naoqi_bridge_msgs` to interface with the Pepper hardware. Configuration for every node is loaded from a YAML file at launch time without recompiling any package. The Behavior Controller acts as the central coordinator, consuming perceptual data from WP4 modules and issuing commands to all WP5 action servers through an XML-defined behavior tree.

Contents

1 Introduction

This document describes the system architecture for the Pepper Robot Tour for Upanzi's Digital Experience Center (DEC). The DEC system is developed as a spin-off of the **CSSR4Africa** (Culturally Sensitive Social Robotics for Africa) project and builds on its ROS1-based software stack, introducing targeted adaptations and new modules to meet the requirements of an autonomous, Pepper-driven tour environment at CMU-Africa. The codebase is maintained in the `pepper4dec` repository and is structured as a ROS2 workspace.

It specifies the architecture's component subsystems, the modules (i.e., ROS2 nodes) comprising each subsystem, and the information exchanged between each module. This includes a specification of the data that are input to each module, the data that are output from each module, and the data that are used to control the operation of the module, including the manner in which this data is made available or accessed: through ROS2 topics, services, or actions, or through configuration and data files.

For each ROS2 node, this deliverable specifies the node's functional behaviour, the configuration parameters that are read from the associated configuration file, the data that are read from an associated input data file (where applicable), the data that are written to an output data file (where applicable), the ROS2 topics to which the node subscribes for input, the ROS2 topics to which the node publishes output, the services that are advertised and served, and the services that are invoked.

In the following, Section 2 states the requirements the architecture must satisfy, Section 3 gives a high-level overview of the system architecture, Section 4 specifies each constituent ROS2 node in detail, and Section 5 presents the detailed architecture showing the topics published by each node, the topics each node subscribes to, and the services and actions advertised and invoked.

2 Requirements Definition

Composability and Modularity

- Each subsystem must be implemented as a collection of independent ROS2 nodes with clearly defined interfaces, enabling modules to be developed, tested, and replaced in isolation.
- All nodes must be independently launchable, and the full system must be startable from a single top-level launch file in the `dec_launch` package.

Communication Standards

- All inter-node communication must use ROS2 topics (for streaming data), services (for request–reply interactions), or actions (for long-running goal-oriented tasks), as appropriate to the interaction pattern.
- Custom message, service, and action types must be defined in the `dec_interfaces` package using `rosidl`.
- Pepper hardware commands must use `naoqi_bridge_msgs` types to interface with the NAOqi driver.

Configuration

- Each node must load all operating parameters from a YAML configuration file supplied via `--params-file` at launch time.
- Topic names that vary by deployment (robot vs. laptop, Pepper camera vs. RealSense) must be resolved from a shared `pepper_topics.yaml` data file rather than hard-coded.

Lifecycle Management

- All primary nodes must implement the ROS2 Managed Node lifecycle (`configure` → `activate` → `deactivate` → `cleanup`).
- Resource-intensive initialisation (model loading, database connections) must occur in `on_configure`; topic subscriptions must be created in `on_activate`.

3 System Architecture Overview

The DEC system architecture comprises the two operational subsystems, WP4 Robot Sensing and WP5 Robot Behaviors, plus supporting infrastructure, as shown in Table 1 and Figure 1. Table 2 enumerates all ROS2 packages, and the ROS2 nodes they provide, in the `pepper4dec` repository; Section 4 gives a detailed specification of each node.

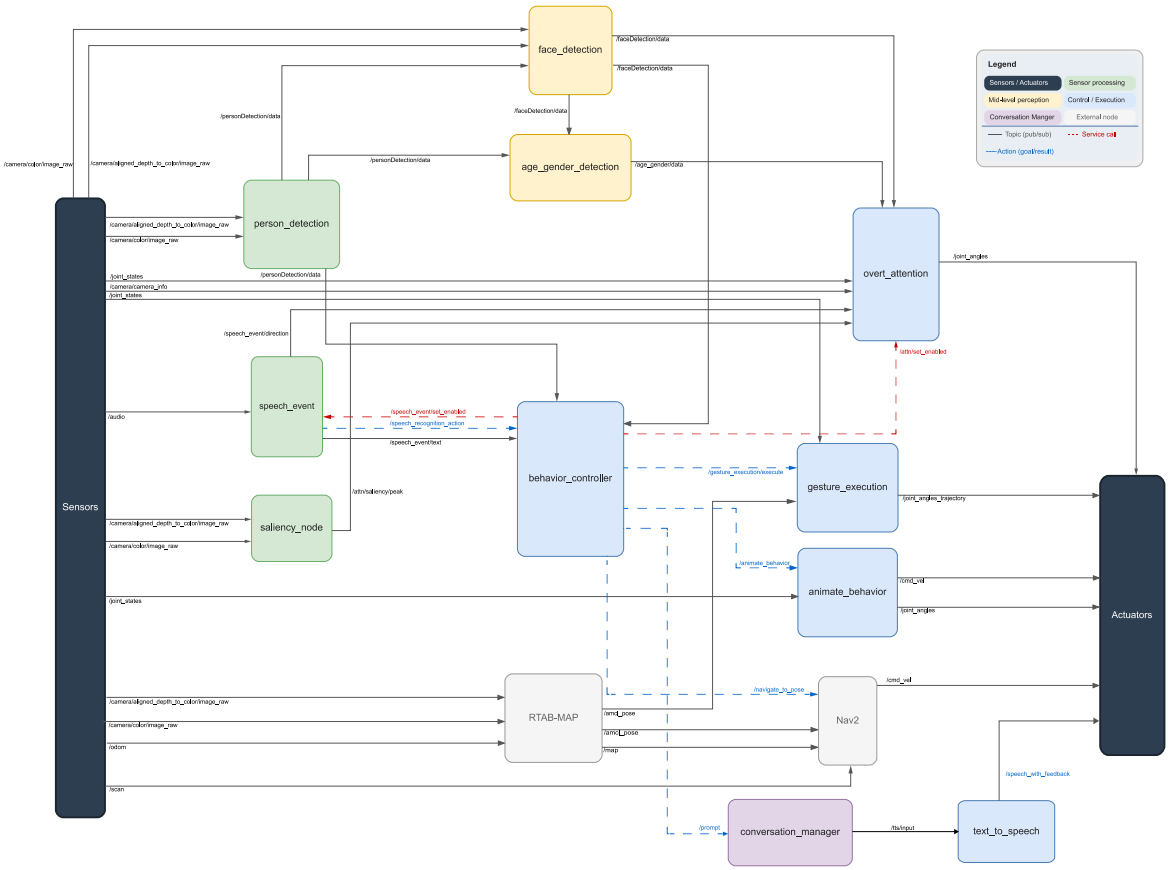

Figure 1: DEC ROS2 system architecture overview diagram showing the Robot Sensing (WP4) and Robot Behaviors (WP5) subsystems, their interconnections, and the data/control flow between modules.

Work Package	Subsystem	Description
WP1	Map & Knowledge Base	DEC environment map, location knowledge base, and cultural knowledge base.
WP2	Interaction Scenario	Use case scenarios, behavior specifications, and the XML behavior tree.
WP3	Systems Engineering	System architecture (this document), software standards, and installation manual.
WP4	Robot Sensing	Person detection, face/gaze/age/gender detection, speech recognition, sound localization, and overt attention.
WP5	Robot Behaviors	Behavior controller, conversation manager, gesture execution, animated behavior, text-to-speech, mapping/localization, and navigation.
WP6	Use Case Evaluation	Demonstration deployment, visitor studies, and evaluation pipelines.

Table 1: DEC system subsystems by work package.

Package	WP	ROS2 Node(s)	Role
person_detection	WP4	person_detection	YOLOv11-based person detection and tracking from RGB-D camera streams.
face_detection	WP4	face_detection age_gender_detection	Face detection, SixDrepNet head-pose/mutual-gaze detection, and MiVOLO-based age/gender estimation.
speech_event	WP4	speech_recognition sound_localization audio_recorder	VAD-based speech recognition with action-server and topic interfaces; sound-source localization; audio logging.
overt_attention	WP4	saliency_node overt_attention attention_visualization	Saliency- and face-driven head-tracking attention controller.
behavior_controller	WP5	behavior_controller	Central mission interpreter; executes the XML behavior tree at 50 Hz via BehaviorTree.CPP v4.
conversation_manager	WP5	conversation_manager	RAG-based dialogue module using ChromaDB + LLM.
gesture_execution	WP5	gesture_action_server	Executes iconic, deictic, bow, and nod gestures via inverse-kinematics-computed joint trajectories.
animate_behavior	WP5	animate_behavior	Manages looping background animated behavior sequences and LED animations.
text_to_speech	WP5	text_to_speech	TTS synthesis with multiple backends (naoqi_ros, Kokoro, ElevenLabs).
pepper_slam	WP5	*(launch/config only)*	Mapping and localization backends (RTAB-Map, SLAM Toolbox, FAST-LIO/Point-LIO) and the Unitree L2 + RealSense sensor-rig TF chain.
pepper_navigation	WP5	*(launch/config only)*	Nav2 stack for global/local path planning, voxel-costmap obstacle avoidance, and a collision-monitor safety layer, over an interchangeable AMCL, FAST-LIO, or RTAB-Map localization profile.
dec_common	Infra	—	Shared C++ utilities consumed by WP4/WP5 nodes: the CameraLifecycleNode base class, the ByteTrack multi-object tracker, and parameter-loading helpers.
dec_interfaces	Infra	—	Custom ROS2 message, service, and action type definitions.
dec_launch	Infra	depth_roi_service	System-level launch files, startup configurations, and hardware utility nodes (depth-ROI service, LiDAR calibration/colorization).

Table 2: ROS2 packages in the pepper4dec repository.

In addition to launch files, dec_launch hosts a small set of standalone utility nodes, including a depth-region-of-interest query service used during camera/LiDAR calibration and by other modules that need on-demand depth statistics:

Service	Type	Description
/get_depth_roi	dec_interfaces/srv/GetDepthROI	Provided by the depth_roi_service node (dec_launch/scripts/depth_roi_service.py). Returns min/max/mean depth statistics for a requested single ROI, an array of points, or an array of ROIs, computed from the current depth image.

Table 3: Infrastructure — Depth ROI Service (dec_launch).

The Behavior Controller (WP5) acts as the central system coordinator. It subscribes to perceptual data from WP4, calls WP5 action servers to execute tour steps, and calls services to control attention, animation, and speech listening modes. All hardware commands reach the Pepper robot via `naoqi_bridge_msgs` topics consumed by the NAOqi driver. The WP4 sensing modules were migrated from ROS1 to ROS2 as part of DEC Deliverable D4.1 (Robot Sensing Migration) [1], building on the original implementations from CSSR4Africa Deliverable D4.2.1 [2], CSSR4Africa Deliverable D4.2.2 [3], and CSSR4Africa Deliverable D4.3.2 [4].

4 ROS Node Specifications

This section specifies each ROS2 node that comprises the DEC system, in alphabetical order of the ROS2 node (or package, for launch-only packages) identifier. For each node we give: its functional specification; the configuration file that determines its operation; the input and output data files it reads and writes (where applicable); the ROS2 topics to which it subscribes and publishes; the services it supports (advertises) and calls; and, where applicable, the actions it supports and calls.

4.1 Animated Behavior

ROS Node Name

`animate_behavior`

Functional Specification

This ROS2 node gives the robot the appearance of an animate agent between tour steps by continually making subtle body movements, flexing its hands, rotating its base, and driving idle LED-ring animations, all while it is enabled by the Behavior Controller. It exposes an action server so that the Behavior Controller can request a specific behavior type (`All`, `body`, `arms`, `hands`, `idle`, `rotation`, or `home`) for a bounded or unbounded duration; a service is also provided to halt the animation immediately. Joint angles are smoothed at ˜30 Hz and published for the NAOqi driver; LED cascade waves are driven via an action client to the NAOqi driver's `run_led` action.

The node can run in normal mode or verbose mode. In verbose mode, diagnostic data is also printed to the terminal.

Configuration File

The operation of the `animate_behavior` node is determined by the contents of a ROS2 parameter YAML file, `animate_behavior_configuration.yaml`, as shown below.

Key	Values	Effect
`verbose_mode`	`true, false`	Specifies whether diagnostic data is to be printed to the terminal.
`led_enabled`	`true, false`	Specifies whether LED cascade animations are driven.
`led_white_step`	`<number>`	Delay in seconds between LED layers rising to white.
`led_dark_step`	`<number>`	Delay in seconds between LED layers going dark.
`led_fade_duration`	`<number>`	Fade duration per LED layer, in seconds.
`led_white_hold`	`<number>`	Seconds to hold all LEDs white before fading.
`led_dark_pause`	`<number>`	Dark pause in seconds before the next LED wave.
`gesture_update_rate`	`<number>`	Animation loop frequency, in Hz.
`gesture_smoothing_factor`	`<number>`	Exponential smoothing coefficient for joint interpolation.
`gesture_motion_speed`	`<number>`	ALMotion speed parameter for animated joint moves.
`gesture_interval_min`	`<number>`	Minimum seconds between new gesture targets.
`gesture_interval_max`	`<number>`	Maximum seconds between new gesture targets.
`gesture_rotation_interval`	`<number>`	Seconds between base-rotation animation changes.

Table 4: Animated Behavior — Configuration file key-value pairs.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

Topic	Message Type	Description
/joint_states	sensor_msgs/ JointState	Current joint positions, used as the starting point for smooth animation interpolation.

Table 5: Animated Behavior — Topics Subscribed.

Topics Published

Topic	Message Type	Description
/joint_angles	naoqi_bridge_msgs/ JointAnglesWithSpeed	Smoothed joint angle commands published at ˜30 Hz to produce idle body animations.
/cmd_vel	geometry_msgs/ Twist	Base velocity commands for rotation-type animations.

Table 6: Animated Behavior — Topics Published.

Services Supported

Service	Type	Effect
/animate_behavior/stop	std_srvs/ Trigger	Immediately halts the active animation and returns the robot to a neutral pose. Called by the behavior_controller node.

Table 7: Animated Behavior — Services Supported.

Services Called

This node does not call any ROS2 services, but it acts as an **action client** to the NAOqi driver's LED action server, as follows.

Action	Type	Effect
/naoqi_driver/run_led	naoqi_bridge_msgs/action/ RunLed	Drives LED cascade wave animations on Pepper's eye rings.

Table 8: Animated Behavior — Actions Called.

Actions Supported

Field	Field Value	Effect
behavior_type	All, body, arms, hands, idle, rotation, home	Selects which animate-behavior category to run.
selected_range	<number> (0--1)	Fraction of the full range of motion to use.
duration_seconds	<number> (0=infinite)	Duration of the animation before the goal completes.

Table 9: Animated Behavior — Action Server /animate_behavior (dec_interfaces /action/AnimateBehavior). Feedback: current_limb, gestures_completed, elapsed_time.

4.2 Behavior Controller

ROS Node Name

behavior_controller

Functional Specification

This ROS2 node is the central system orchestrator, playing the role of the CSSR4Africa Robot Mission Interpreter. It is a ROS2 Lifecycle Node that loads an XML behavior tree at startup and ticks it at 50 Hz once activated, using BehaviorTree.CPP v4 with the behaviortree_ros2 bridge. A companion plain rclcpp::Node (behavior_controller_bt) is used for all BT action and service clients. The dynamics of the interaction between the robot and the visitor are specified by the behavior tree, which recruits the WP4 sensing nodes and WP5 behavior nodes by calling services, subscribing to topics, and sending action goals, as detailed below. Live tree visualization is supported via Groot2. The node can run in normal mode or verbose mode.

The registered BT node types fall into four groups. Action nodes that send goals to the WP4/WP5 action servers: AnimateBehavior, Gesture, Navigate, SpeechRecognition, ConversationManager, SpeechWithFeedback, and TTS. Service-calling control nodes: StopAnimateBehavior, SetOvertAttention, and SetSpeechListening. Perception condition nodes backed by topic subscriptions: CheckFaceDetected, IsVisitorDiscovered, IsMutualGazeDiscovered, ListenForSpeech, GetVisitorResponse, and IsVisitorResponseYes. Knowledge-base and blackboard utility nodes: RetrieveListOfExhibits, IsListWithExhibit, SelectExhibit, PopExhibitFromList, SetBlackboardValue, CheckBlackboard, and LogEvent.

Configuration File

The operation of the behavior_controller node is determined by the contents of a configuration file, behavior_controller_configuration.yaml, as shown below.

Key	Values	Effect
scenario_specification	<specificationFile>	Specifies the name of the XML behavior-tree file to load.
culture_knowledge_base	cultureKnowledgeBase.yaml	Specifies the filename of the cultural knowledge base.
environment_knowledge_base	decEnvironmentKnowledgeBase_short.yaml	Specifies the filename of the environment knowledge base.
verbose_mode	true, false	Specifies whether diagnostic data is to be printed to the terminal.

Table 10: Behavior Controller — Configuration file key-value pairs.

Input Data File

This node reads the XML behavior-tree file named by scenario_specification, and the culture and environment knowledge base YAML files named by culture_knowledge_base and environment_knowledge_base (see Helper Classes Used, below).

Output Data File

This node does not write to an output data file.

Topics Subscribed

Topic	Message Type	Description
/face_detection/data	dec_interfaces/FaceDetection	Visitor presence and mutual-gaze status, consumed by the CheckFaceDetected, IsVisitorDiscovered, and IsMutualGazeDiscovered BT nodes.
/speech_event/text	std_msgs/String	Transcribed visitor utterances, consumed by the ListenForSpeech and GetVisitorResponse BT nodes.

Table 11: Behavior Controller — Topics Subscribed.

Topics Published

This node does not publish to any topics.

Services Supported

This node does not provide and advertize any services.

Services Called

Service	Type	Effect
/animate_behavior/stop	std_srvs/Trigger	Immediately halt the current background animation (StopAnimateBehavior BT node).
/overt_attention/set_enabled	std_srvs/SetBool	Enable or disable the overt attention system (SetOvertAttention BT node).
/speech_event/set_enabled	std_srvs/SetBool	Mute or unmute speech recognition during TTS playback (SetSpeechListening BT node).

Table 12: Behavior Controller — Services Called.

Actions Supported

This node does not provide and advertize any actions.

Actions Called

This node acts as an action client, sending goals to the action servers detailed in the table below. Each entry specifies the action server, the BT node type that sends the goal, and the expected effect.

Action	Type	Effect
/speech_recognition	dec_interfaces/action/SpeechRecognition	Listen for visitor speech for up to wait seconds (SpeechRecognition BT node).
/conversation_manager	dec_interfaces/action/ConversationManager	Submit visitor utterance for RAG-based response and intent classification (ConversationManager BT node).
/text_to_speech	dec_interfaces/action/TTS	Synthesise and play a speech utterance, blocking until playback completes (TTS BT node).
/animate_behavior	dec_interfaces/action/AnimateBehavior	Start a looping background animation for the specified duration (AnimateBehavior BT node).
/gesture_execution	dec_interfaces/action/Gesture	Execute a named gesture (iconic, deictic, bow, nod) (Gesture BT node).
/navigate_to_pose	nav2_msgs/action/NavigateToPose	Send a navigation goal to the Nav2 stack (Navigate BT node).
/naoqi_driver/naoqi_driver/speech_with_feedback	naoqi_bridge_msgs/action/SpeechWithFeedback	Direct NAOqi ALAnimatedSpeech playback with word-timing feedback and automatic body-language gestures; the primary path for speaking the Conversation Manager's response (SpeechWithFeedback BT node).

Table 13: Behavior Controller — Actions Called.

Helper Classes Used

This node uses an instantiation of a helper class, KnowledgeManager (a singleton), to read the culture knowledge base file and the environment knowledge base file named in the configuration file, and to retrieve tour waypoints, gesture targets, and speech phrases at runtime using class access methods.

4.3 Conversation Manager

ROS Node Name

`conversation_manager`

Functional Specification

This ROS2 node provides natural dialogue capability through a Retrieval-Augmented Generation (RAG) pipeline backed by ChromaDB and an OpenAI-compatible LLM. When the Behavior Controller submits a visitor utterance as an action goal, the module retrieves relevant context from the Upanzi knowledge base and streams the LLM response internally, then returns the full response text, classified intent, and confidence score as the action result once generation completes. The module does not publish to any ROS2 topic; the Behavior Controller passes the returned response text directly to the `SpeechWithFeedback` BT node (NAOqi ALAnimatedSpeech), which interprets embedded prosody tags and drives contextual gestures while speaking. The node can run in normal mode or verbose mode, and is a ROS2 lifecycle node.

Configuration File

The operation of the `conversation_manager` node is determined by the contents of a configuration file, `converation_manager_configuration.yaml`, as shown below.

Key	Values	Effect
`llm.base_url`	`<url>`	Specifies the OpenAI-compatible LLM API endpoint.
`llm.model`	`<modelid>`	Specifies the LLM model to use (the API key is read from an environment variable, not this file).
`embedding.model`	`<modelid>`	Specifies the sentence-embedding model used to index and query the knowledge base.
`retrieval.mode`	`rag`, `full_context`	`rag`: vector search over ChromaDB, only relevant snippets sent to the LLM. `full_context`: skip vector search, send the entire knowledge base every turn.
`search.similarity_threshold`	`<number>`	Minimum similarity score for a retrieved snippet to be included (`rag` mode only).
`search.top_k`	`<number>`	Maximum number of snippets retrieved per query (`rag` mode only).
`conversation.max_history_turns`	`<number>`	Maximum number of conversation turns stored in history.
`conversation.context_turns`	`<number>`	Number of recent turns included in each LLM request.
`conversation.max_response_sentences`	`<number>`	Caps the LLM's `max_tokens` (`max_response_sentences` × 50 + 512).
`data.default_path`	`<path>`	Specifies the path to the Upanzi knowledge base data file.
`debug.verbose`	`true`, `false`	Specifies whether diagnostic data is to be printed to the terminal.

Table 14: Conversation Manager — Configuration file key-value pairs.

Input Data File

This node reads the Upanzi knowledge base data file named by `data.default_path` and indexes it into a ChromaDB vector store on first use.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node does not subscribe to any topics.

Topics Published

This node does not publish to any topics; its only external interface is the `/conversation_manager` action server described below.

Services Supported

This node does not provide and advertize any services.

Services Called

This node does not call any services.

Actions Supported

Action Server	Type	Description
`/conversation_manager`	`dec_interfaces/action/ConversationManager`	Goal: `prompt` (visitor utterance). Result: `response`, `intent`, `confidence`. Feedback: `status` (`searching` / `generating`). Called by the `behavior_controller` node.

Table 15: Conversation Manager — Action Server.

4.4 Face and Mutual Gaze Detection

ROS Node Name

`face_detection`, and a second, independent lifecycle node in the same package, `age_gender_detection`.

Functional Specification

The `face_detection` node performs real-time multi-face detection, SixDrepNet-based head-pose estimation, and mutual-gaze evaluation, always running its face detector on the full RGB-D camera frame. When `require_person_detection` is enabled, it additionally subscribes to the person-detection stream and matches each detected face to a tracked person via Hungarian assignment,

inheriting that person's tracking ID and using it to prioritise the face's depth source; when disabled, faces are tracked directly by the node's own ByteTrack instance instead. For each detected face it reports a tracking ID, pixel-space centroid, bounding box, and a boolean mutual-gaze flag, published as an array of records.

The `age_gender_detection` node performs age and gender estimation using the MiVOLO ONNX model (a six-channel face-plus-body input, distinct from the SixDrepNet model used for head-pose/gaze). Estimation is queued for a tracked person only once mutual gaze is observed within a configurable depth range; the node fuses a face crop from `face_detection`'s output with a body crop from `person_detection`'s output and the raw camera image. Estimates are smoothed over a rolling per-person history and periodically refreshed rather than reported every frame. Results are published as a JSON-encoded string per person (label ID, age, gender, gender confidence, estimation count, bounding box) on a dedicated topic rather than as fields on the `FaceDetection` message.

Both nodes can run in normal mode or verbose mode. In verbose mode, diagnostic data is also printed to the terminal and, for `face_detection`, output images are displayed in an OpenCV window.

Configuration File

The operation of the `face_detection` node is determined by the contents of a configuration file, `face_detection_configuration.yaml`, as shown below.

Key	Values	Effect
`use_compressed`	`true, false`	Specifies to use compressed image or raw image topics.
`camera`	`realsense, pepper, video`	Specifies which camera source to use.
`verbose_mode`	`true, false`	Specifies whether diagnostic data is to be printed and images displayed.
`image_timeout`	`<number>`	Timeout in seconds for image reception.
`sixdrepnet_confidence`	`<number>`	Face detection confidence threshold for the SixDrepNet algorithm.
`sixdrepnet_headpose_angle`	`<number>`	Head-pose angle threshold (degrees) for mutual-gaze classification.
`require_person_detection`	`true, false`	Specifies whether faces are cropped from person-detection bounding boxes, or detected independently.
`person_detection_timeout`	`<number>`	Timeout in seconds for person-detection messages.
`prioritize_face_depth`	`true, false`	Specifies whether to prefer face-region depth over person-region depth when both are available.

Table 16: Face Detection — Configuration file key-value pairs.

The `age_gender_detection` node reads a second configuration file, `age_gender_detection_configuration.yaml`, as shown below.

Key	Values	Effect
device	cuda, cpu	Specifies whether the MiVOLO ONNX model runs on GPU or CPU.
face_only	true, false	Specifies whether to use a face-only model input (the packaged model requires face+body, so this is false by default).
max_cache_age_sec	<number>	Maximum allowed gap between an image and a bounding-box timestamp for them to be paired.
min_estimate_interval_sec	<number>	Rate limit between successive estimates for the same tracked person.
re_estimate_interval_sec	<number>	Interval after which a person's age/gender estimate is refreshed.
max_depth_m	<number>	Estimation is only triggered for persons within this depth, in metres.

Table 17: Age/Gender Detection — Configuration file key-value pairs.

Input Data File

Neither node reads from an input data file (model weights are loaded from a fixed path within the package share directory, not treated as a configurable data file).

Output Data File

Neither node writes to an output data file.

Topics Subscribed

Topic	Message Type	Node	Description
/person_detection/data	dec_interfaces/ PersonDetection	face_detection, age_gender_detection	Person bounding boxes used to crop face/body regions.
/camera/color/image_raw_custom	sensor_msgs/Image	face_detection	RGB image stream (resolved from pepper_topics.yaml).
/camera/aligned_depth_to_color/ image_raw_custom	sensor_msgs/Image	face_detection	Aligned depth image for metric face distance.
/camera/color/image_raw	sensor_msgs/Image	age_gender_detection	Raw camera frame used to extract face/-body crops.
/face_detection/data	dec_interfaces/ FaceDetection	age_gender_detection	Face centroids, bounding boxes, and mutual-gaze flags used to gate estimation.

Table 18: Face Detection — Topics Subscribed.

Topics Published

Topic	Message Type	Description
/face_detection/data	dec_interfaces/ FaceDetection	Per-frame list of detected faces with tracking IDs, centroids, bounding boxes, and mutual-gaze flags.
/face_detection/debug	sensor_msgs/Image	Debug visualisation with face bounding boxes (face_detection, verbose mode).
/face_detection/depth_debug	sensor_msgs/Image	Depth debug visualisation (face_detection, verbose mode).
/face_detection/age_gender_results	std_msgs/String	Per-person JSON-encoded age/gender estimate (label ID, age, gender, gender confidence, estimation count, bounding box), published by age_gender_detection.

Table 19: Face Detection — Topics Published.

Services Supported

Neither node provides and advertizes any services.

Services Called

Neither node calls any services.

Helper Classes Used

Both nodes are built on dec_common's CameraLifecycleNode helper base class, which provides the shared camera lifecycle behaviour (subscription setup, compressed/raw image handling, and timeout logic) common to all WP4 camera-consuming nodes.

4.5 Gesture Execution

ROS Node Name

gesture_action_server

Functional Specification

This ROS2 node computes inverse-kinematics-based joint trajectories for Pepper's arms, hands, and body to produce expressive gestures. It exposes a single action server that accepts a gesture type (iconic, deictic, bow, or nod), gesture name, duration, and an optional 3D target coordinate for deictic pointing. Joint trajectories are published for the NAOqi driver. The module also subscribes to current joint positions and to the fused robot pose for deictic target resolution: the map→base_footprint pose published by lio_localization's transform_fusion node in the FAST-LIO localization profile (Section 4.7); yaw is derived from its orientation quaternion. The node can run in normal mode or verbose mode.

Configuration File

The operation of the `gesture_action_server` node is determined by the contents of a configuration file, `gesture_execution_configuration.yaml`, as shown below.

Key	Values	Effect
`gestureDescriptors`	`gesture.yaml`	Specifies the filename of the file in which gesture descriptors (joint-angle waypoints per named gesture) are stored.
`robotTopics`	`pepperTopics.yaml`	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
`verboseMode`	`true`, `false`	Specifies whether diagnostic data is to be printed to the terminal.

Table 20: Gesture Execution — Configuration file key-value pairs.

Input Data File

This node reads the gesture descriptor file named by `gestureDescriptors`, which defines the joint-angle waypoints for each named iconic and symbolic gesture.

Output Data File

This node does not write to an output data file.

Topics Subscribed

Topic	Message Type	Description
`/joint_states`	`sensor_msgs/JointState`	Current joint positions used as the starting point for trajectory generation.
`/localization`	`nav_msgs/Odometry`	Fused `map`→`base_footprint` robot pose from `lio_localization`'s `transform_fusion` (FAST-LIO localization profile), used to resolve 3D deictic target coordinates into joint angles.

Table 21: Gesture Execution — Topics Subscribed.

Topics Published

Topic	Message Type	Description
`/joint_angles_trajectory`	`naoqi_bridge_msgs/JointAnglesTrajectory`	Joint trajectory commands sent to Pepper's arms and body via the NAOqi driver.
`/gesture_execution/visualization`	`visualization_msgs/Marker`	RViz marker for debugging deictic target positions.

Table 22: Gesture Execution — Topics Published.

Services Supported

This node does not provide and advertize any services.

Services Called

This node does not call any services.

Actions Supported

Field	Field Value	Effect
gesture_type	iconic, symbolic, deictic, bow, nod	Selects the gesture category.
gesture_name	<string>	Names the specific gesture descriptor (iconic/symbolic only).
gesture_duration	<number> (ms)	Duration of the gesture.
bow_nod_angle	<number> (degrees)	Bow/nod inclination angle (bow/nod only).
location_x, location_y, location_z	<number> (metres)	Target coordinates for deictic pointing.

Table 23: Gesture Execution — Action Server /gesture_execution (dec_interfaces/action/Gesture). Feedback: elapsed_seconds. Called by the behavior_controller node.

4.6 Overt Attention

ROS Node Name

saliency_node, overt_attention, and attention_visualization.

Functional Specification

This trio of ROS2 nodes controls Pepper's head-tracking behavior using a saliency-driven attention system. The saliency_node computes bottom-up visual saliency peaks from the camera stream. The overt_attention node (a Lifecycle node) fuses face detection data, those saliency peaks, and camera geometry to compute target head-joint angles, which are published for the NAOqi driver; the Behavior Controller enables and disables it via a service. The attention_visualization node renders the fused saliency and attention state for debugging. All three nodes can run in normal mode or verbose mode.

Configuration File

The operation of all three nodes is determined by the contents of a single configuration file, overt_attention_configuration.yaml, as shown below.

Key	Values	Effect
Shared (all nodes)		
use_compressed	true, false	Specifies to use compressed image or raw image topics.
camera_type	pepper, realsense	Specifies which camera to use.
saliency_node		
publish_map	true, false	Specifies whether the saliency map is published for visualization.
use_depth_weighting	true, false	Specifies whether saliency is weighted by depth.
depth_min_m, depth_max_m	<number>	Depth range, in metres, used for depth weighting.

continued on next page

Table 24 – *continued from previous page*

Key	Values	Effect
depth_weight_min	<number>	Minimum depth-weighting factor applied at depth_max_m.
min_peak	<number>	Minimum saliency value to be considered a peak.
num_peaks	<number>	Maximum number of saliency peaks reported per frame.
process_hz	<number>	Saliency computation frequency, in Hz.
overt_attention		
start_enabled	true, false	Specifies whether attention is active on startup.
move_to_default_on_disable	true, false	Specifies whether the head returns to the default pose when disabled.
default_yaw, default_pitch	<number>	Default head-joint angles, in radians.
default_move_speed	<number>	Speed used when moving to the default pose.
saliency_yaw_lim, saliency_pitch_up, saliency_pitch_dn	<number>	Joint limits, in radians, when tracking a saliency peak. (The equivalent face-tracking limits are fixed in code, not configurable.)
face_timeout	<number>	Seconds before a lost face target is discarded.
engaged_priority_bonus	<number>	Priority bonus applied to a mutually-gazing face.
face_switch_cooldown	<number>	Minimum seconds between switching tracked faces.
prefer_closer_faces	true, false	Specifies whether nearer faces are prioritised.
max_face_distance	<number>	Maximum face distance, in metres, considered for tracking.
min_angular_change_deg	<number>	Dead-zone: skip a head-move command if already within this angle.
target_smoothing_alpha	<number>	EMA weight for the new target angle (0 = frozen, 1 = raw).
saliency_min_score, saliency_min_cooldown, saliency_max_dwell	<number>	Minimum score, cooldown, and maximum dwell time for saliency targets.
switch_score_ratio	<number>	Minimum score ratio required to switch to a new saliency target.
same_target_threshold_deg	<number>	Angular threshold, in degrees, to treat two saliency peaks as the same target.
enable_ior	true, false	Enables the Inhibition-of-Return mechanism.
ior_max_suppression, ior_half_life, ior_radius_deg	<number>	IOR suppression strength, decay half-life, and radius. (Cleanup threshold and maximum tracked-location count are fixed in code, not configurable.)
attention_visualization		
show_face_ids	true, false	Specifies whether each tracked face's ID is overlaid on the debug image.

continued on next page

Table 24 – *continued from previous page*

Key	Values	Effect
`show_depth`	`true`, `false`	Specifies whether each tracked face's depth estimate is overlaid.

Table 24: Overt Attention — Configuration file key-value pairs.

Input Data File

None of the three nodes read from an input data file.

Output Data File

None of the three nodes write to an output data file.

Topics Subscribed

Node	Topic	Message Type	Description
saliency_node	/camera/color/image_raw (or compressed)	sensor_msgs/Image / CompressedImage	RGB camera stream for saliency computation; topic resolved from pepper_topics.yaml per camera_type.
saliency_node	/camera/aligned_depth_to_color/image_raw (or compressed)	sensor_msgs/Image / CompressedImage	Depth stream used for depth-weighted saliency when use_depth_weighting is enabled.
overt_attention	/face_detection/data	dec_interfaces/ FaceDetection	Face centroids and gaze flags used to compute head-tracking targets.
overt_attention	/overt_attention/saliency_peak	std_msgs/ Float32MultiArray	Visual saliency peak co-or-

Topics Published

Topic	Message Type	Description
/joint_angles	naoqi_bridge_msgs/ JointAnglesWithSpeed	Target yaw and pitch joint angles commanded to Pepper's head via the NAOqi driver.
/overt_attention/target_angles	geometry_msgs/ Vector3	Computed target angles published for monitoring and visualization.
/overt_attention/saliency_peak	std_msgs/ Float32MultiArray	Top-N salient peak locations, published by saliency_node.
/overt_attention/saliency_map/compressed	sensor_msgs/ CompressedImage	Annotated saliency map image, published by saliency_node when publish_map is true.
/overt_attention/visualization	sensor_msgs/Image	Annotated debug overlay, published by attention_visualization.

Table 26: Overt Attention — Topics Published.

Services Supported

Service	Type	Effect
/overt_attention/set_enabled	std_srvs/ SetBool	Enable or disable the attention system. When disabled, the head returns to the default pose. Called by the behavior_controller node.

Table 27: Overt Attention — Services Supported.

Services Called

This node does not call any services.

4.7 Path Planning and Navigation

ROS Node Name

pepper_navigation (package). This is a launch/configuration-only package: it does not build its own compiled ROS2 nodes, but launches and configures the third-party Nav2 stack.

Functional Specification

This package integrates **Nav2** (ROS2 Navigation Stack) for global/local path planning and obstacle avoidance, run on top of one of three interchangeable localization profiles built on pepper_slam (Section 4.8), as summarised in Table 28. The Behavior Controller sends navigation goals via the Nav2 BasicNavigator API, which calls the standard /navigate_to_pose action server; pre-built maps and RTAB-Map databases are stored in pepper_navigation/map/.

Launch File	Localization Method	Map Source
pepper_nav2_amcl.launch.py	AMCL over a 2D /scan (flattened from the L2's 3D point cloud via pointcloud_to_laserscan) matched against a static occupancy grid.	.pgm/.yaml occupancy grid (default pepper_map_lc_clean.yaml).
pepper_nav2_fastlio_loc.launch.py	FAST-LIO odometry with prior-map ICP registration (lio_localization); the lightest-weight profile, with neither RTAB-Map nor pose-graph optimization running.	Prior .pcd point cloud for ICP, plus a .pgm/.yaml occupancy grid for the global costmap's static layer.
pepper_nav2_rtabmap_loc.launch.py	RTAB-Map in localization mode (Mem/IncrementalMemory=false) over FAST-LIO odometry; RTAB-Map publishes /map directly.	RTAB-Map .db database.

Table 28: Nav2 Localization Profiles (pepper_navigation).

All three profiles use nav2_costmap_2d::VoxelLayer (per-voxel clearing) rather than a flat obstacle layer for both the local and global costmaps, replacing an earlier configuration that left stale obstacle marks behind moving visitors and could register Pepper's own low-mounted LiDAR rig as an obstacle. The local costmap combines a self-hit- and ground-plane-filtered LiDAR point cloud with the RealSense camera's forward-facing point cloud as a complementary dense obstacle source. A nav2_collision_monitor node is inserted as a safety layer in the velocity command path: controller_server and behavior_server publish candidate velocity commands to cmd_vel_raw, which the collision monitor gates against a self-hit-filtered LiDAR point cloud before republishing the final /cmd_vel consumed by the robot base. Two circular polygons are configured identically across all three profiles: a 0.40 m stop polygon and a 0.80 m slow-down polygon (30% of commanded speed).

Configuration File

Each localization profile reads its own Nav2 parameter YAML file: nav2_params_amcl.yaml, nav2_params_fastlio_loc.yaml, or nav2_params_rtabmap_loc.yaml, configuring the costmap, controller, planner, and collision-monitor plugins summarised above. A fourth, legacy file, nav2_params.yaml, is read by the older pepper_navigation.launch.py bringup, which predates the collision-monitor safety layer and additionally configures a keepout-zone costmap filter (map/keepout_zone.yaml) that has not been carried over to the three current profiles.

Input Data File

This package reads the occupancy-grid map, prior point cloud, or RTAB-Map database named in Table 28, according to the active localization profile.

Output Data File

This package does not write to an output data file.

Topics Subscribed

Topic	Message Type	Description
/points	sensor_msgs/PointCloud2	Raw 3D point cloud from the Unitree L2, the primary voxel-costmap obstacle source (FAST-LIO-loc / RTAB-Map-loc profiles).
/points_costmap	sensor_msgs/PointCloud2	Self-hit- and ground-plane-filtered L2 point cloud, the voxel-costmap obstacle source for the AMCL profile.
/points_safety	sensor_msgs/PointCloud2	Self-hit-filtered L2 point cloud consumed by the collision monitor.
/camera/depth/color/points	sensor_msgs/PointCloud2	RealSense point cloud, secondary local-costmap obstacle source.
/scan	sensor_msgs/LaserScan	2D laser scan for AMCL, produced from /points by pointcloud_to_laserscan.
/map	nav_msgs/OccupancyGrid	Occupancy grid, served by nav2_map_server (AMCL / FAST-LIO-loc) or published directly by RTAB-Map (RTAB-Map-loc).

Table 29: Path Planning and Navigation — Topics Subscribed.

Topics Published

Topic	Message Type	Description
cmd_vel_raw	geometry_msgs/Twist	Candidate velocity commands from controller_server/behavior_server, gated by the collision monitor.
/cmd_vel	geometry_msgs/Twist	Final, safety-gated velocity command published by the collision monitor and consumed by Pepper's mobile base.
/tf	tf2_msgs/TFMessage	Transform tree including map→odom (from the active localization backend) and the levelled odom/map frames published by pepper_slam.

Table 30: Path Planning and Navigation — Topics Published.

Services Supported

This package does not provide and advertize any services.

Services Called

This package does not call any services.

Actions Supported

Action Server	Type	Description
/navigate_to_pose	nav2_msgs/action/NavigateToPose	Standard Nav2 action server. Accepts a geometry_msgs/PoseStamped goal; returns on arrival or failure. Called by the behavior_controller node via BasicNavigator.

Table 31: Path Planning and Navigation — Action Server.

4.8 Mapping and Localization

ROS Node Name

`pepper_slam` (package). This is a launch/configuration/utility-script-only package: the SLAM/odometry engines it wraps (RTAB-Map, SLAM Toolbox, FAST-LIO, Point-LIO) are third-party ROS2 packages.

Functional Specification

This package provides mapping and localization for a Unitree L2 3D LiDAR + Intel RealSense D435i sensor rig mounted on Pepper's base, described by `urdf/pepper_sensor_rig.urdf.xacro` and rooted at the `base_footprint` frame independently of the NAOqi body description. Three interchangeable backends are provided, summarised in Table 32: **RTAB-Map** (RGB-D SLAM/localization), **SLAM Toolbox** (2D LiDAR SLAM over a flattened `/scan`), and **FAST-LIO/Point-LIO** (lidar-inertial odometry over the L2 point cloud), the last of which also supports lightweight real-time localization by ICP-registering live scans against a saved point cloud (`lio_localization`) without RTAB-Map running.

Launch File	Backend	Description
`rtabmap_base.launch.py`	RTAB-Map	RGB-D SLAM/localization.
`slam_toolbox.launch.py`	SLAM Toolbox	2D LiDAR SLAM over a flattened `/scan`.
`fastlio_odometry.launch.py` / `pointlio_odometry.launch.py`	FAST-LIO / Point-LIO	Lidar-inertial odometry over the L2 point cloud (mapping mode).

Table 32: Mapping and Localization — Launch Files.

All static sensor-rig transforms (Pepper base to LiDAR, LiDAR to camera, and the camera's internal frames) are published from a single configuration file, `config/sensor_tf.yaml`, via a dedicated static-publisher script or an equivalent URDF/`robot_state_publisher` launch path.

Configuration File

Configuration is spread across several YAML files rather than a single one, summarised in Table 33.

File	Effect
config/sensor_tf.yaml	Source of truth for the static sensor-rig transform chain (Pepper base → LiDAR → camera and internal camera frames).
config/ekf_lio_wheel.yaml	Configures the robot_localization EKF node used by ekf_fusion.launch.py to fuse leveled LIO odometry with wheel odometry.
config/mapper_params_online_async.yaml	Configures the slam_toolbox async_slam_toolbox_node.
config/record_qos.yaml	QoS overrides used when recording/replaying bag files.

Table 33: Mapping and Localization — Configuration files.

Input Data File

The FAST-LIO/Point-LIO localization launch files read a prior-map point cloud (.pcd) for ICP registration, and the URDF/xacro sensor-rig description.

Output Data File

The mapping launch files (RTAB-Map, SLAM Toolbox, FAST-LIO/Point-LIO mapping mode) write map data (an RTAB-Map .db database, an occupancy grid, or a point-cloud map, respectively) to disk, consumed by pepper_navigation.

Topics Subscribed

This package's topic subscriptions vary by launch file (backend); see the individual launch files for details. In all backends, the Unitree L2 point cloud (/points) and, where applicable, the RealSense RGB-D streams are the primary sensor inputs.

Topics Published

Topic	Message Type	Description
/odometry/lio_leveled	nav_msgs/Odometry	Leveled LIO odometry, published by leveled_odometry_publisher.py, consumed by the EKF fusion node.
/odometry/pepper_odom_relabeled	nav_msgs/Odometry	Wheel odometry republished on the odom TF frame, by pepper_odom_relabel.py.
/odometry/filtered	nav_msgs/Odometry	EKF-fused odometry (ekf_fusion.launch.py only; does not publish TF).
/tf, /tf_static	tf2_msgs/TFMessage	The sensor-rig static transform chain, plus the dynamic odom→base_footprint and map→odom edges from the active LIO/SLAM backend.

Table 34: Mapping and Localization — Topics Published.

Services Supported

This package does not provide and advertize any services.

Services Called

This package does not call any services.

4.9 Person Detection

ROS Node Name

`person_detection`

Functional Specification

This ROS2 node performs real-time detection and tracking of visitors using a YOLOv11 ONNX model and a ByteTrack multi-object tracker applied to RGB-D image streams. The camera source (RealSense or Pepper's built-in camera) and whether to use compressed topics are resolved from `pepper_topics.yaml` at runtime. Each detected person is assigned a persistent tracking ID and described by centroid pixel coordinates, metric depth, bounding box dimensions, and detection confidence, published as an array of records. The node can run in normal mode or verbose mode.

Configuration File

The operation of the `person_detection` node is determined by the contents of a configuration file, `person_detection_configuration.yaml`, as shown below.

Key	Values	Effect
`camera`	`pepper, realsense`	Specifies which camera source to use.
`use_compressed`	`true, false`	Specifies to use compressed image or raw image topics.
`image_timeout`	`<number>`	Timeout in seconds for image reception.
`verbose_mode`	`true, false`	Specifies whether diagnostic data is to be printed to the terminal.
`confidence_threshold`	`<number>`	Minimum YOLO detection confidence threshold (0.0–1.0).
`target_classes`	`<list>`	Object classes to detect and track (e.g. `person`).
`track_threshold`	`<number>`	ByteTrack detection-confidence threshold for starting a new track.
`track_buffer`	`<number>`	Number of frames to keep a lost track before removing it.
`match_threshold`	`<number>`	IoU threshold for matching detections to existing tracks.
`frame_rate`	`<number>`	Expected frame rate of the video stream, in fps.

Table 35: Person Detection — Configuration file key-value pairs.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

Topic	Message Type	Description
`/camera/color/image_raw_custom`	`sensor_msgs/Image`	RGB image stream (resolved from `pepper_topics.yaml`).
`/camera/aligned_depth_to_color/image_raw_custom`	`sensor_msgs/Image`	Aligned depth image stream for metric depth estimation.

Table 36: Person Detection — Topics Subscribed.

Topics Published

Topic	Message Type	Description
/person_detection/data	dec_interfaces/ PersonDetection	Per-frame list of detected persons with tracking IDs, centroids, bounding boxes, confidence scores, and depth.
/person_detection/debug	sensor_msgs/Image	Debug visualisation image with bounding box overlays.
/person_detection/depth_debug	sensor_msgs/Image	Depth debug visualisation image.

Table 37: Person Detection — Topics Published.

Services Supported

This node does not provide and advertize any services.

Services Called

This node does not call any services.

Helper Classes Used

This node is built on dec_common's CameraLifecycleNode helper base class (shared camera lifecycle behaviour) and uses dec_common's ByteTrack tracker implementation for multi-person tracking.

4.10 Speech Event

ROS Node Name

speech_recognition, sound_localization, and audio_recorder, all in the speech_event package.

Functional Specification

The speech_recognition node provides VAD-based speech recognition with a post-VAD noise-reduction stage. It subscribes to the Pepper microphone audio stream, applies a voice-activity detector to isolate speech segments, and transcribes them using a Whisper-based ASR backend. Two interfaces are exposed to downstream consumers: a ROS2 action server for blocking goal-based listening requests from the Behavior Controller, and, when configured for standalone operation, a publisher for continuous topic-based consumption. A service allows the Behavior Controller to mute the recognizer during TTS playback.

The sound_localization node performs SRP-PHAT beamforming over the same microphone array to estimate the azimuth (horizontal-plane direction of arrival) of a detected sound.

The audio_recorder node is a utility that logs raw microphone audio to disk for debugging and dataset collection.

All three nodes are launched together and can run in normal mode or verbose mode. Of the three, only speech_recognition is a ROS2 lifecycle node; sound_localization and audio_recorder are plain nodes.

Configuration File

The operation of all three nodes is determined by the contents of a single configuration file, `speech_event_configuration.yaml`, as shown below.

Key	Values	Effect
speech_recognition		
microphone_topic	<topic>	ROS topic publishing naoqi_bridge_msgs/AudioBuffer.
input_sample_rate, sample_rate	<number> (Hz)	Robot's native microphone rate, and the target rate for VAD/Whisper.
device, compute_type	cuda/cpu, float16/float32	PyTorch device and computation precision for Whisper.
language	<ISO639-1code>	Transcription language, e.g. en.
whisper_model_id	<modelid/path>	HuggingFace model ID or local path for the ASR model.
speech_threshold, neg_threshold, min_silence_duration_ms	<number>	Two-threshold VAD hysteresis: probability to start speech, probability below which silence accumulates, and silence duration to end a segment.
min_speech_duration, max_speech_duration_s, pre_speech_buffer_ms	<number>	Minimum/maximum segment duration and pre-speech lookback buffer.
intensity_threshold	<number>	RMS amplitude gate below which audio is skipped before VAD.
transcription_timeout_s	<number>	Maximum time to wait for Whisper to finish transcribing a segment.
action_server	true, false	true: audio gated by an action goal (result returned as transcription). false: continuous processing, transcript published to a topic.
vad_always_active	true, false	When true, VAD runs (and publishes speech probability) even without an active ASR goal, enabling barge-in monitoring during TTS.
noise_cleaning_enabled, noise_profile_path, noise_alpha	true/false, <path>, <number>	Enables and configures the post-VAD bandpass/notch/Wiener noise-cleaning pipeline.
sound_localization		
microphone_topic, sample_rate	<topic>, <number>	Audio input topic and sample rate.
speed_of_sound, nfft, angular_resolution, freq_range_min, freq_range_max	<number>	SRP-PHAT beamforming parameters (pyroomacoustics).
num_chunks_for_localization, update_rate_hz	<number>	Minimum audio chunks accumulated per localization run, and the maximum update frequency.

continued on next page

Table 38 – *continued from previous page*

Key	Values	Effect
confidence_threshold, intensity_threshold	<number>	Minimum confidence to publish a result, and the RMS amplitude gate.
enable_smoothing, smoothing_window	true/false, <number>	Enables and sizes a circular-mean temporal smoothing window over azimuth history.
audio_recorder		
mic_topic	<topic>	Audio input topic.
output_base	<path>	Base path for the recorded WAV output file(s).
max_seconds	<number>	Stop recording after this many seconds (0 = until interrupted).
split_channels	true, false	Also write one mono WAV file per microphone channel.

Table 38: Speech Event — Configuration file key-value pairs.

Input Data File

None of the three nodes read from an input data file (the optional noise profile named by `noise_profile_path` is treated as part of the noise-cleaning configuration, not a separate data-file interface).

Output Data File

The `audio_recorder` node writes recorded audio to the WAV file(s) named by `output_base`.

Topics Subscribed

Topic	Message Type	Description
/naoqi_driver/audio	naoqi_bridge_msgs/AudioBuffer	Raw microphone audio stream from the Pepper robot, subscribed to by all three nodes (topic configurable via parameter).

Table 39: Speech Event — Topics Subscribed.

Topics Published

Topic	Message Type	Description
/speech_event/text	std_msgs/String	Transcribed visitor utterance, published after VAD isolation (standalone mode only).
/speech_event/vad_speech_prob	std_msgs/Float32	Real-time VAD speech probability for monitoring and barge-in detection.
/sound_detection/direction	std_msgs/Float32	Estimated azimuth of a detected sound, from sound_localization.

Table 40: Speech Event — Topics Published.

Services Supported

Service	Type	Effect
/speech_event/set_enabled	std_srvs/ SetBool	Enable or disable the speech recognizer (called by the behavior _controller node before and after TTS playback).

Table 41: Speech Event — Services Supported.

Services Called

This node does not call any services.

Actions Supported

Action Server	Type	Description
/speech_recognition	dec_interfaces/action/ SpeechRecognition	Goal: wait (max seconds to listen). Result: transc ription (empty string if timeout). Feedback: status (waiting / speech / transcribing). Called by the behavior_controller node.

Table 42: Speech Event — Action Server.

4.11 Text-to-Speech

ROS Node Name

text_to_speech

Functional Specification

This ROS2 node converts text to speech and plays it through the Pepper robot's speakers. Multiple synthesis backends are supported: naoqi_ros (publishes plain text for the NAOqi driver's on-board synthesis), kokoro_local, kokoro_pepper, elevenlabs_local, and elevenlabs_p epper. The active backend and playback method are selected via the configuration file. In addition to the action server, the node subscribes to a topic for incremental sentence-by-sentence playback and publishes a boolean topic while audio is playing. The action server blocks until playback is fully complete, making it safe to sequence in a behavior tree. Barge-in via VAD is honoured: the node monitors the VAD speech-probability topic, and if the visitor speaks during playback, remaining sentences are dropped and the action returns success. The node can run in normal mode or verbose mode.

Configuration File

The operation of the text_to_speech node is determined by the contents of a configuration file, text_to_speech_configuration.yaml, as shown below.

Key	Values	Effect
engine	naoqi_ros, kokoro_local, kokoro_pepper, elevenlabs_local, elevenlabs_pepper	Specifies the synthesis + playback backend.
playback_method	file, stream	For *_pepper backends: file SCPs a WAV to the robot for full-feedback playback; stream sends raw PCM chunks with no SCP required.
naoqi_speech_topic	/speech	Topic used by the naoqi_ros backend.
chars_per_second, speech_padding_s	<number>	Estimated speech rate and padding used to time naoqi_ros playback completion.
voice, sample_rate, output_device	<string>, <number>, <index>	Kokoro backend voice, sample rate, and output audio device.
elevenlabs_api_key	<string>	ElevenLabs API key (normally set via the ELEVENLABS_API_KEY environment variable instead).
elevenlabs_voice_id, elevenlabs_model	<string>	ElevenLabs voice and model selection.
elevenlabs_stability, elevenlabs_similarity_boost, elevenlabs_style, elevenlabs_speed	<number>	ElevenLabs voice-generation tuning parameters.
stream_volume	<number>	Output volume multiplier for stream playback method.

Table 43: Text-to-Speech — Configuration file key-value pairs.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

Topic	Message Type	Description
/text_to_speech/input	std_msgs/String	Sentences enqueued for incremental, sentence-by-sentence playback (typically fed by an LLM response stream).
/speech_event/vad_speech_prob	std_msgs/Float32	VAD speech probability, monitored for barge-in during playback.

Table 44: Text-to-Speech — Topics Subscribed.

Topics Published

Topic	Message Type	Description
/speech	std_msgs/String	Plain-text utterances published to the NAOqi driver for on-board synthesis (naoqi_ros backend only).
/text_to_speech/speaking	std_msgs/Bool	True while Pepper is actively speaking, false otherwise (all backends).

Table 45: Text-to-Speech — Topics Published.

Services Supported

This node does not provide and advertize any services.

Services Called

This node does not call any services, but acts as a **service and action client** to the naoqi_driver node for the kokoro_pepper / elevenlabs_pepper backends, as follows.

Service / Action	Type	Description
/naoqi_driver/load_audio_file	naoqi_bridge_msgs/srv/LoadAudioFile	Service client. Uploads synthesised WAV audio to the robot (file playback method).
/naoqi_driver/unload_audio_file	naoqi_bridge_msgs/srv/UnloadAudioFile	Service client. Frees an uploaded audio file on the robot after playback.
/naoqi_driver/send_audio_buffer	naoqi_bridge_msgs/srv/SendAudioBuffer	Service client. Streams raw PCM audio buffers to the robot (stream playback method, low latency).
/naoqi_driver/play_audio	naoqi_bridge_msgs/action/PlayAudio	Action client. Plays an uploaded audio file on Pepper's speakers (file playback method).

Table 46: Text-to-Speech — Services and Action Used (kokoro_pepper / elevenlabs_pepper backends).

Actions Supported

Action Server	Type	Description
`/text_to_speech`	`dec_interfaces/action/TTS`	Goal: `text` (string, may contain multiple sentences). Result: `success`, `message`. Feedback: `status` (`queuing` / `speaking`). Called by the `behavior_controller` node.

Table 47: Text-to-Speech — Action Server.

5 System Architecture in Detail

Figure 1 (Section 3) shows the DEC system architecture at the level of ROS2 nodes and packages, their interconnecting topics, and the sensors and actuators at the edges of the system. It is the DEC counterpart of the CSSR4Africa system architecture diagram: the `behavior_controller` node occupies the same central-coordinator role as CSSR4Africa's `behaviorController`, recruiting the WP4 sensing nodes and WP5 action/navigation nodes documented in Section 4 by subscribing to their topics, calling their services, and sending goals to their action servers, exactly as detailed in each node's specification above.

References

[1] DEC Deliverable D4.1: Robot Sensing Migration. Technical report, Carnegie Mellon University Africa, 2026. Revision 1.0. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D4.1.pdf.

[2] CSSR4Africa Deliverable D4.2.1: Person Detection and Localization. Technical report, Carnegie Mellon University Africa, 2025. Revision 2.1. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D4.2.1.pdf.

[3] CSSR4Africa Deliverable D4.2.2: Face & Mutual Gaze Detection and Localization. Technical report, Carnegie Mellon University Africa, 2025. Revision 1.5. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D4.2.2.pdf.

[4] CSSR4Africa Deliverable D4.3.2: Speech Event. Technical report, Carnegie Mellon University Africa, 2025. Revision 1.5. Available online: https://cssr4africa.github.io/deliverables/CSSR4Africa_Deliverable_D4.3.2.pdf.

Principal Contributors

The main authors of this deliverable are as follows (in alphabetical order).

Yohannes Haile, Carnegie Mellon University Africa.

Document History

Version 1.0

First draft.
Yohannes Haile.
28 February 2026.

Version 1.1

Updated to reflect actual ROS2 package structure from the `pepper4dec` repository.
Added Overt Attention, Animated Behavior, and Text-to-Speech module specifications.
Yohannes Haile.
01 May 2026.

Version 1.2

All interface tables corrected from verified source code.
Updated topic names: `/joint_angles`, `/joint_angles_trajectory`, `/speech_event/text`, `/attn/set_enabled`, `/speech_event/set_enabled`.
Updated message types: `naoqi_bridge_msgs/JointAnglesTrajectory`, `naoqi_bridge_msgs/JointAnglesWithSpeed`.
Updated action servers: `/gesture_execution/execute`, `animate_behavior`, `/tts`, `/speech_recognition_action`.
Updated service names and camera topics throughout.
Yohannes Haile.
01 June 2026.

Version 1.3

Second cross-check against the current `pepper4dec` source.
Renamed node identifiers to match actual ROS2 names (`personDetection`→`person_detection`, `faceDetection`→`face_detection`, `speechEvent`→`speech_recognition`/`sound_localization`/`audio_recorder`, `overtAttention`→`saliency_node`/`unified_attention_node`/`attention_visualization`, `gestureExecution`→`gesture_action_server`, `animateBehavior`→`animate_behavior`, `textToSpeech`→`text_to_speech`, `conversationManager`→`conversation_manager`, `behaviorController`→`behavior_controller`).
Corrected topic names throughout.
Corrected action server names, and the SpeechWithFeedback action type/server name.
Removed the fabricated `/tts/input` topic from the Conversation Manager spec; it publishes no topics.
Yohannes Haile.
04 July 2026.

Version 1.4

Third cross-check against the current `pepper4dec` source.
Corrected the navigation package name throughout (`pepper_navmap`→`pepper_navigation`).
Added the previously undocumented `pepper_odom_anchor` package and its `robot_localization` node (later removed in v1.5; see below).
Completed the Text-to-Speech interface tables.

Documented the previously-omitted `dec_launch /get_depth_roi` service.
Added the missing `bow_nod_angle` goal field to the Gesture Execution action table.
Removed the unused, legacy `ConversationManagerPrompt.srv` and `ObjectDetection.msg` interfaces from `dec_interfaces` (dead code, never wired to any node).
Yohannes Haile.
08 July 2026.

Version 1.5

Split Navigation and Localization into Mapping and Localization (`pepper_slam`: RTAB-Map, SLAM Toolbox, FAST-LIO/Point-LIO) and Path Planning and Navigation (`pepper_navigation`: AMCL/FAST-LIO/RTAB-Map profiles, voxel costmaps, new collision-monitor safety layer).
Removed the `pepper_odom_anchor` package and all its references, following its deletion from the `pepper4dec` source tree.
Corrected the implementation-language label for five WP4/WP5 packages mislabelled Python instead of C++, and added the previously undocumented `dec_common` infrastructure package.
Rewired Gesture Execution's robot-pose subscription to the FAST-LIO-fused `/localization` topic, replacing the dead `/robot_localization/pose`.
Restructured the document to the CSSR4Africa D3.1 template: one subsection per ROS2 node instead of the previous WP4/WP5 subsystem sections.
Added full Configuration File key-value tables for every node, sourced from each package's actual ROS2 parameter YAML file.
Yohannes Haile.
06 August 2026.

Version 1.6

Fixed the configuration tables that overflowed or drifted from their headings (non-floating placement and `xltabular` pagination) and the stray backslash before underscores in path names, shaded the per-node divider rows, and moved the launch-file names into a summary table.
Renamed the attention-controller node `unified_attention_node` to `overt_attention`, removed the internal frame-naming contract from the Mapping and Localization spec, corrected the Face Detection specification (the detector runs on the full RGB-D frame, with person detections matched to faces afterward), and noted the lifecycle-node status of `speech_recognition` and `ConversationManagerNode`.
Yohannes Haile.
07 August 2026.

Version 1.7

Synced the Overt Attention section with the codebase's parameter cleanup (35 to 29 tunable parameters): removed the saliency node's `down_w`/`down_h`/`overlay_alpha`/`peak_min_distance_px`, the attention controller's face-tracking joint limits and `same_face_threshold_deg`, and the IOR cleanup/max-locations pair, all of which are now fixed in code rather than YAML-configurable (or, for `same_face_threshold_deg`, removed entirely as a parameter that was never actually read). Corrected the `attention_visualization` block from its previous `publish_overlay`/`publish_markers`/`show_metrics` (never read by the node) to the real `show_face_ids`/`show_depth`. Also added several previously undocumented topics: the saliency and visualization nodes' own camera/depth subscriptions, and the saliency map image and visualization overlay publishers.

Yohannes Haile.
08 August 2026.

Version 1.8

Synced with further `pepper4dec` commits: added Conversation Manager's `conversation.max_response_sentences` and Speech Event's `transcription_timeout_s`, both declared/read in code with defaults but previously missing from their shipped YAML and this document alike. Renamed `pepper_slam`'s `fastlio_mapping.launch.py`/`pointlio_mapping.launch.py` to `fastlio_odometry.launch.py`/`pointlio_odometry.launch.py`, matching the codebase rename.
Yohannes Haile.
13 August 2026.

Version 1.9

Reformatted the Behavior Controller node specification to match the layout used by every other node section (replaced the non-standard "Action Goals Sent" subsection with the standard **Actions Supported** and **Actions Called** pair, and renamed the "Action Server" column to "Action"), and corrected it against the `behavior_controller` source: completed the list of registered behavior-tree node types, which previously copied an outdated log line and omitted the exhibit, blackboard, and visitor-condition nodes; and fixed the Topics Subscribed consumers (`/face_detection/data` is read by `CheckFaceDetected`, `IsVisitorDiscovered`, and `IsMutualGazeDiscovered`, and `/speech_event/text` by `ListenForSpeech` and `GetVisitorResponse`).
Yohannes Haile.
15 August 2026.